

**Міністерство освіти і науки України
Національний університет «Львівська політехніка»
Інститут інформаційно-комунікаційних технологій та
електронної інженерії**



**Звіт з лабораторної роботи №1
«Застосування принципів SOLID об'єктно-орієнтованого програмування
при проєктуванні програмних систем у сфері телекомунікацій»
з дисципліни «Об'єктно-орієнтоване програмування»**

Підготував:
ст. групи ІХ-21
Ярошенко Георгій

Прийняла:
Плесканка М.В.

Львів 2026р.

Мета роботи: Ознайомитися з принципами SOLID. Навчитися застосовувати їх під час проєктування класів на Python. Формувати навички створення гнучкого, масштабованого та підтримуваного коду.

Короткі теоретичні відомості

SOLID — це набір із п'яти принципів об'єктно-орієнтованого програмування, запропонованих Робертом Мартіном, які спрямовані на покращення якості архітектури програмного забезпечення.

S — Single Responsibility (Єдина відповідальність): Кожен клас має відповідати лише за щось одне. Тобто у нього має бути лише одна причина для внесення змін у майбутньому.

O — Open/Closed (Відкритість/Закритість): Програмний код має бути відкритим для розширення (додавання нового функціоналу), але закритим для модифікації (зміни існуючого, вже робочого коду).

L — Liskov Substitution (Підстановка Лісков): Дочірні класи (підкласи) повинні вміти повноцінно замінювати свій базовий клас. При такій заміні логіка роботи програми не повинна зламатися.

I — Interface Segregation (Розділення інтерфейсу): Краще створювати кілька вузькоспеціалізованих інтерфейсів, ніж один великий "універсальний". Класи не повинні залежати від методів, якими вони не користуються.

D — Dependency Inversion (Інверсія залежностей): Модулі верхніх та нижніх рівнів не повинні залежати один від одного напряму. Уся взаємодія має будуватися через абстракції.

Результати виконаної роботи

Завдання 1. Single Responsibility Principle (SRP)

1.1

Є клас `CallReport`, який формує звіт про дзвінки та зберігає його у файл.

Розділити відповідальності відповідно до SRP.

1.2

Клас `Subscriber`:

- зберігає дані абонента
- відправляє SMS
- розраховує баланс

Перепроєктувати систему так, щоб кожен клас мав одну відповідальність.

Результат виконання завдання див. рис. 1 та рис. 2.

```
1 # Ярошенко Георгій
2
3 # 1.1 Розділення класу CallReport
4 class CallReportGenerator:
5     # Формує звіт про дзвінки
6     def generate(self, calls_data):
7         return "Звіт згенеровано"
8
9 class CallReportSaver:
10     # Зберігає готові дані у файл
11     def save(self, report, filename):
12         print(f"Звіт збережено у {filename}")
13
14 # 1.2 Розділення класу Subscriber
15 class SubscriberData:
16     # Зберігає виключно дані абонента
17     def __init__(self, phone_number):
18         self.phone_number = phone_number
19
20 class SmsSender:
21     # Відповідає за відправку SMS
22     def send(self, subscriber_data, message):
23         print(f"Повідомлення на {subscriber_data.phone_number}: {message}")
24
25 class BalanceCalculator:
26     # Розраховує баланс окремо від інших процесів
27     def calculate(self, subscriber_data):
28         return 150.0
```

Рис. 1 – Код до завдання 1

The screenshot displays a code editor with a file explorer on the left and a console at the bottom. The file explorer shows a project named 'SRP' with files: .gitignore, JSP, CACHEDIR.TAG, DIP.py, ISP.py, LSP.py, OCP.py, pyvenv.cfg, and SRP.py. The code editor shows Python code for a system that generates a call report and sends an SMS. The console output shows the execution of the code, including the generation of a call report and the sending of an SMS.

```
30 # Перевірка завдання
31 if __name__ == "__main__":
32     print("--- Завдання 1.1: Call Report ---")
33     report_gen = CallReportGenerator()
34     report_saver = CallReportSaver()
35
36     report_content = report_gen.generate("дані_дзвінків_за_місяць")
37     print(report_content)
38     report_saver.save(report_content, filename="call_report_01.txt")
39
40
41     print("\n--- Завдання 1.2: Subscriber ---")
42     subscriber = SubscriberData("+380991234567")
43
44     sms = SmsSender()
45     sms.send(subscriber, message="Вітаємо у нашій мережі!")
46
47     calc = BalanceCalculator()
48     balance = calc.calculate(subscriber)
49     print(f"Поточний баланс: {balance} грн")
```

SRP x

C:\Users\AYAYAKing\Documents\LabHordBub\00P\2Kurs2026\00PLabs2026\Lab01\.venv\Scripts\python.exe C:\Users\AYAYAKing\Documents\

--- Завдання 1.1: Call Report ---

Звіт згенеровано

Звіт збережено у call_report_01.txt

--- Завдання 1.2: Subscriber ---

Повідомлення на +380991234567: Вітаємо у нашій мережі!

Поточний баланс: 150.0 грн

Рис. 2 – Перевірка коду з виведенням результату у консоль

Завдання 2. Open/Closed Principle (OCP)

2.1

Реалізувати систему тарифікації дзвінків з базовим тарифом.

Додати новий тариф **без зміни існуючого коду**.

2.2

Система підтримує тарифи: VoiceTariff, DataTariff.

Розширити систему тарифом RoamingTariff, не змінюючи логіку розрахунку вартості. Результат виконання завдання див. рис. 3 та рис. 4.

```

1      # Ярошенко Георгій
2
3      from abc import ABC, abstractmethod
4
5      # 2.1 Базовий клас для системи тарифікації
6      class Tariff(ABC):
7          @abstractmethod
8          def calculate_price(self, usage):
9              pass
10
11     # Існуючі тарифи (Voice та Data)
12     class VoiceTariff(Tariff):
13         def calculate_price(self, minutes):
14             return minutes * 2.0 # Логіка для дзвінків
15
16     class DataTariff(Tariff):
17         def calculate_price(self, megabytes):
18             return megabytes * 0.1 # Логіка для інтернету
19
20     # 2.2 Розширення системи тарифом RoamingTariff
21     class RoamingTariff(Tariff):
22         def calculate_price(self, minutes):
23             return minutes * 15.0 # Новий тариф додано без зміни старого коду

```

Рис. 3 – Код до завдання 2

The screenshot shows a Python IDE with a file named 'SRP.py'. The code in the editor defines three classes: `VoiceTariff`, `DataTariff`, and `RoamingTariff`, all inheriting from an abstract class `Tariff`. The `__main__` block creates instances of these classes and prints the results of their `calculate_price` methods for specific inputs.

```

25     # Перевірка виконання коду
26     if __name__ == "__main__":
27         print("--- Завдання 2: ОCP ---")
28         voice = VoiceTariff()
29         data = DataTariff()
30         roaming = RoamingTariff()
31
32         print(f"Вартість 10 хв розмови: {voice.calculate_price(10)} грн")
33         print(f"Вартість 50 МБ інтернету: {data.calculate_price(50)} грн")
34         print(f"Вартість 10 хв у роумінгу: {roaming.calculate_price(10)} грн")

```

The output window shows the following results:

```

--- Завдання 2: ОCP ---
Вартість 10 хв розмови: 20.0 грн
Вартість 50 МБ інтернету: 5.0 грн
Вартість 10 хв у роумінгу: 150.0 грн

```

Рис. 4 – Код перевірки та результат виконання коду

3. Liskov Substitution Principle (LSP)

3.1

Є базовий клас `NetworkConnection`.

Реалізувати підкласи `LTEConnection`, `WiFiConnection`, які можна взаємозамінно використовувати.

3.2

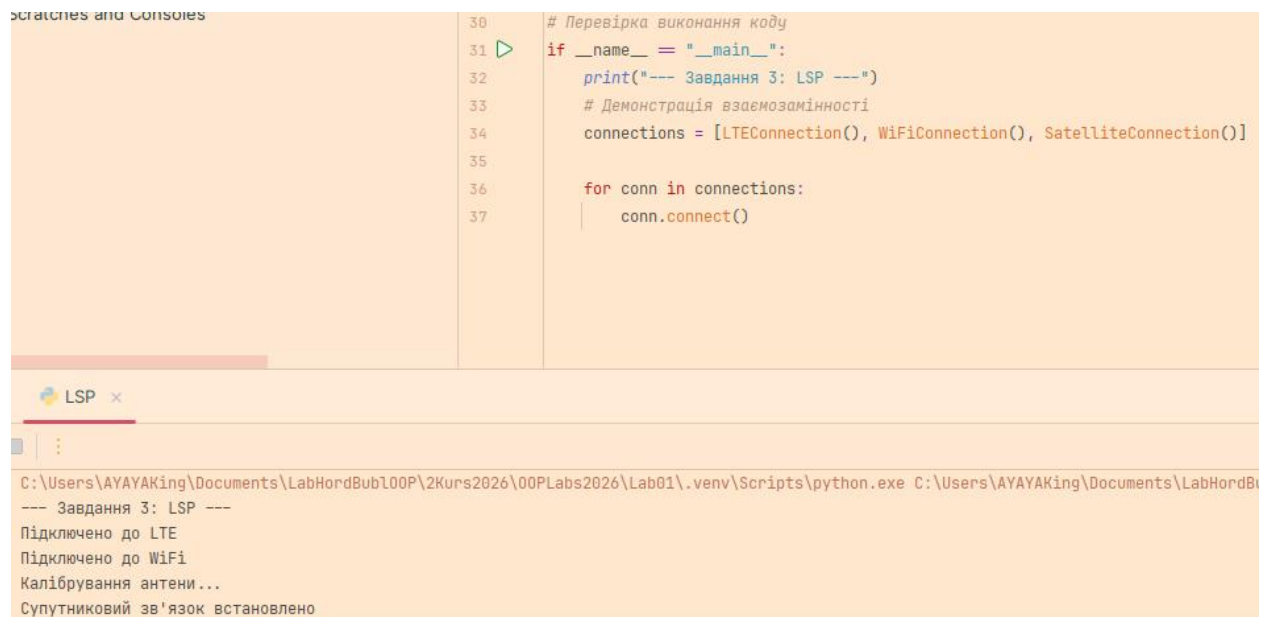
Виправити створену ієрархію, де клас `SatelliteConnection` порушує очікувану поведінку базового класу (супутник не може працювати як звичайне з'єднання). Результат виконання завдання див. рис. 5 та рис. 6.

```

1      # Ярошенко Георгій
2
3      from abc import ABC, abstractmethod
4
5      class NetworkConnection(ABC):
6          @abstractmethod
7          def connect(self):
8              pass
9
10     # 3.1 Реалізація взаємозамінних підкласів
11     class LTEConnection(NetworkConnection):
12         def connect(self):
13             print("Підключено до LTE")
14
15     class WiFiConnection(NetworkConnection):
16         def connect(self):
17             print("Підключено до WiFi")
18
19     # 3.2 Виправлена ієрархія для SatelliteConnection
20     class SatelliteConnection(NetworkConnection):
21         def _calibrate_dish(self):
22             # Специфічна логіка прихована від клієнта
23             print("Калібрування антени...")
24
25         def connect(self):
26             # Базовий метод відпрацьовує очікувано
27             self._calibrate_dish()
28             print("Супутниковий зв'язок встановлено")

```

Рис. 5 – Код до завдання 3



```
30 # Перевірка виконання коду
31 if __name__ == "__main__":
32     print("--- Завдання 3: LSP ---")
33     # Демонстрація взаємозамінності
34     connections = [LTEConnection(), WiFiConnection(), SatelliteConnection()]
35
36     for conn in connections:
37         conn.connect()
```

LSP x

```
C:\Users\AYAYAKing\Documents\LabHordBub100P\2Kurs2026\00PLabs2026\Lab01\.venv\Scripts\python.exe C:\Users\AYAYAKing\Documents\LabHordB
--- Завдання 3: LSP ---
Підключено до LTE
Підключено до WiFi
Калібрування антени...
Супутниковий зв'язок встановлено
```

Рис. 6 – Код перевірки та результат виконання коду

Завдання 4. Interface Segregation Principle (ISP)

4.1

Інтерфейс TelecomDevice має методи:

- make_call()
- send_sms()
- connect_to_network()

Розділити інтерфейс відповідно до ISP.

4.2

IoT-пристрій у мережі оператора повинен лише передавати дані.

Спроекувати систему інтерфейсів без зайвих методів. Результат виконання завдання див. рис. 7 та рис. 8.


```

1      # Ярошенко Георгій
2
3      from abc import ABC, abstractmethod
4
5      # 4.1 Розділення універсального інтерфейсу
6      class ICallable(ABC):
7          @abstractmethod
8          def make_call(self): pass
9
10     class ISmsSender(ABC):
11         @abstractmethod
12         def send_sms(self): pass
13
14     class INetworkConnectable(ABC):
15         @abstractmethod
16         def connect_to_network(self): pass
17
18     # Смартфон реалізує всі необхідні йому можливості
19     class Smartphone(ICallable, ISmsSender, INetworkConnectable):
20         def make_call(self): print("Дзвінок...")
21         def send_sms(self): print("Відправка SMS...")
22         def connect_to_network(self): print("Підключення до мережі...")
23
24     # 4.2 IoT-пристрій імплементує лише передачу даних
25     class IoTDevice(INetworkConnectable):
26         def connect_to_network(self):
27             print("Передача даних з датчика у мережу")
28

```

Рис. 7 – Код до завдання 4

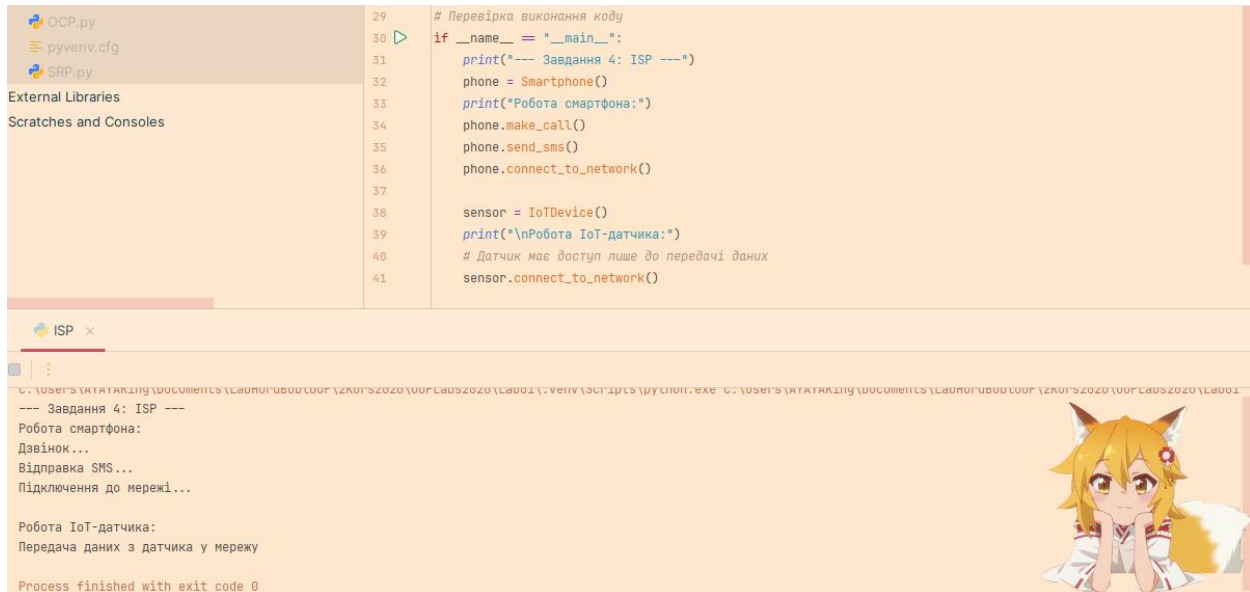


Рис. 8 – Код перевірки та результат виконання коду

Завдання 5. Dependency Inversion Principle (DIP)

5.1

Система моніторингу мережі напряму використовує FileLogger.

Перепроєктувати систему відповідно до DIP.

5.2

Реалізувати можливість підключення:

- FileLogger
- ServerLogger
- ConsoleLogger

(без змін у коді класу NetworkMonitor). Результат виконання завдання див. рис. 9 та рис. 10.

```

1  # Ярошенко Георгій
2
3  from abc import ABC, abstractmethod
4
5  # 5.1 Абстракція логера
6  class Logger(ABC):
7      @abstractmethod
8      def log(self, message): pass
9
10 # 5.2 Варіанти підключення логерів
11 class FileLogger(Logger):
12     def log(self, message):
13         print(f"Запис у файл: {message}")
14
15 class ServerLogger(Logger):
16     def log(self, message):
17         print(f"Відправка на сервер: {message}")
18
19 class ConsoleLogger(Logger):
20     def log(self, message):
21         print(f"Вивід у консоль: {message}")
22
23 # Клас моніторингу не потребує змін при зміні логера
24 class NetworkMonitor:
25     def __init__(self, logger: Logger):
26         self.logger = logger # Залежність інжектована через конструктор
27
28     def check(self):
29         self.logger.log("ycnix")

```

Рис. 9 – Код до завдання 5



```
31 # Перевірка коду
32 if __name__ == "__main__":
33     print("---- Завдання 5: DIP ----")
34
35     # Монітор з логуванням у файл
36     file_monitor = NetworkMonitor(FileLogger())
37     file_monitor.check()
38
39     # Монітор з логуванням на сервер
40     server_monitor = NetworkMonitor(ServerLogger())
41     server_monitor.check()
42
43     # Монітор з виводом у консоль
44     console_monitor = NetworkMonitor(ConsoleLogger())
45     console_monitor.check()
```

DIP x

```
C:\Users\AYAYAKing\Documents\LabHordBub100P\2Kurs2026\00PLabs2026\Lab01\venv\Scripts\python.exe C:\Users\AYAYAKing\Documents\LabHordBub100P\2Kurs2026\Lab01\
--- Завдання 5: DIP ---
Запис у файл: успіх
Відправка на сервер: успіх
Вивід у консоль: успіх

Process finished with exit code 0
```



Рис. 10 – Код перевірки та результат виконання коду

Висновок

У ході виконання лабораторної роботи було успішно вивчено та застосовано на практиці принципи SOLID при проєктуванні об'єктно-орієнтованих систем мовою Python. На прикладі моделей з телекомунікаційної сфери закріплено навички рефакторингу та створення гнучкого, масштабованого й підтримуваного коду. Розділення відповідальностей, використання абстракцій та побудова правильних ієрархій дозволили зробити програмні класи більш незалежними та відкритими до безпечного розширення.